# EDUCATION INNOVATION CAFE (EIC)

**Nom complet :** Françoise NOEL

**Titre du devoir :** Assistant interactif sur Wokwi

**Cours :** IA & Robotique

**Professeur :** Carlos Yfrazin

**Date : 09 octobre 2026**

# 1. Présentation du devoir

Ce devoir consiste à réaliser, sur le simulateur Wokwi, un assistant interactif capable d'exécuter des commandes tapées au clavier. L'utilisateur écrit une commande dans le moniteur série ; la carte Arduino la reçoit, l'analyse, exécute l'action correspondante et affiche une confirmation sur un écran LCD.

Il s'agit de la première étape d'un projet plus large : ici, les commandes sont écrites. La reconnaissance vocale et l'intelligence artificielle seront ajoutées dans une étape ultérieure, ce qui permettra de passer d'une commande tapée à une commande parlée.

## 1.1 Objectifs

- Concevoir et câbler un circuit électronique complet dans un simulateur.
- Commander plusieurs actionneurs (LED, servomoteur, buzzer) avec une carte Arduino.
- Communiquer avec la carte grâce au moniteur série (communication série).
- Analyser une chaîne de caractères reçue pour en déduire une action.
- Fournir un retour visuel à l'utilisateur avec un écran LCD.
- Documenter le travail dans un rapport PDF clair et complet.

## 1.2 Ce qui est demandé

| Élément | Description |
|---|---|
| Carte | Arduino Mega (ou ESP32) et un écran LCD. |
| LED | Trois LED : rouge, verte et jaune, chacune avec sa résistance. |
| Servomoteur | Un servomoteur pouvant se positionner à 0°, 90° ou 180°. |
| Buzzer | Un buzzer pouvant sonner ou s'arrêter. |
| Bouton | Un bouton qui éteint les LED et arrête le buzzer. |
| Commandes | Saisies dans le moniteur série : allumer/éteindre chaque LED, positionner le servo, faire sonner/arrêter le buzzer. |
| Confirmation | Chaque action est confirmée sur l'écran LCD. |

# 2. Principe de fonctionnement

Le fonctionnement repose sur une boucle simple en quatre temps :

1. **Réception :** l'utilisateur tape une commande dans le moniteur série (par exemple ROUGE ON) et appuie sur Entrée.
2. **Lecture :** la carte lit le texte reçu jusqu'au retour à la ligne.
3. **Analyse :** le programme compare le texte avec la liste des commandes connues.
4. **Action et confirmation :** le composant concerné est activé, puis un message de confirmation s'affiche sur le LCD et dans le moniteur série.

En parallèle, la carte surveille en permanence le bouton. Si on appuie dessus, toutes les LED s'éteignent et le buzzer s'arrête, quelle que soit la commande en cours.

### 2.1 Le moniteur série

Le moniteur série est une fenêtre de communication entre l'ordinateur (ou le simulateur) et la carte Arduino. Dans ce devoir, il joue deux rôles : il envoie les commandes à la carte, et il affiche les messages de retour envoyés par la carte. La vitesse de communication est fixée à 9600 bauds avec Serial.begin(9600).

### 2.2 La liste des commandes

| Commande | Effet |
|---|---|
| ROUGE ON / ROUGE OFF | Allume / éteint la LED rouge. |
| VERTE ON / VERTE OFF | Allume / éteint la LED verte. |
| JAUNE ON / JAUNE OFF | Allume / éteint la LED jaune. |
| SERVO 0 | Positionne le servomoteur à 0°. |
| SERVO 90 | Positionne le servomoteur à 90°. |
| SERVO 180 | Positionne le servomoteur à 180°. |
| BUZZER ON | Fait sonner le buzzer. |
| BUZZER OFF | Arrête le buzzer. |
| Bouton (physique) | Éteint les trois LED et arrête le buzzer. |

Le programme accepte les majuscules et les minuscules, car il convertit la commande en majuscules avant de la comparer. Toute commande non reconnue provoque un message « Commande inconnue ».

## 3. Liste des composants

| Composant | Quantité | Rôle |
|---|---|---|
| Arduino Mega | 1 | Cerveau du système : exécute le programme. |
| Écran LCD 1602 I2C | 1 | Affiche les confirmations (2 lignes de 16 caractères). |
| LED rouge | 1 | Témoin lumineux commandé. |
| LED verte | 1 | Témoin lumineux commandé. |
| LED jaune | 1 | Témoin lumineux commandé. |
| Résistance 220 Ω | 3 | Limite le courant dans chaque LED pour la protéger. |
| Servomoteur | 1 | Se positionne à un angle précis (0°, 90°, 180°). |
| Buzzer | 1 | Émet un son lorsqu'il est activé. |
| Bouton poussoir | 1 | Éteint les LED et arrête le buzzer. |

# 4. Câblage détaillé

Sur Wokwi, un fil se crée en cliquant sur une broche, puis sur la broche à relier. Un clic sur le fil permet d'en changer la couleur (rouge pour le 5V, noir pour la masse, une autre couleur pour les signaux).

## 4.1 Tableau de câblage

| Composant | Broche du composant | Relié à |
|---|---|---|
| LED rouge | Anode (via 220 Ω) / Cathode | D2 / GND |
| LED verte | Anode (via 220 Ω) / Cathode | D3 / GND |
| LED jaune | Anode (via 220 Ω) / Cathode | D4 / GND |
| Buzzer | + / − | D5 / GND |
| Servomoteur | PWM / V+ / GND | D9 / 5V / GND |
| Bouton | Broche gauche / broche droite | D7 / GND |
| LCD I2C | SDA / SCL / VCC / GND | Broche 20 / Broche 21 / 5V / GND |

## 4.2 Explications pas à pas

1. **Alimentation :** relier une broche 5V de la Mega à la ligne (+) de la breadboard, et une broche GND à la ligne (−). Tous les composants prennent ensuite leur alimentation sur ces lignes.
2. **LED :** pour chacune, relier la broche numérique (D2, D3 ou D4) à une résistance de 220 Ω, la résistance à l'anode de la LED, puis la cathode à GND. Chaque LED a sa propre résistance.
3. **Buzzer :** broche + sur D5, broche − sur GND.
4. **Servomoteur :** fil de signal sur D9, fil V+ sur 5V, fil GND sur GND.
5. **Bouton :** une broche sur D7, l'autre sur GND. Aucune résistance externe n'est nécessaire : le programme active la résistance interne (INPUT_PULLUP).
6. **LCD I2C :** GND sur GND, VCC sur 5V, SDA sur la broche 20 et SCL sur la broche 21. Ces broches sont propres à l'Arduino Mega (elles diffèrent de celles de l'Arduino Uno).

## 4.3 Vérifications avant de lancer la simulation

- Chaque LED possède sa résistance et son anode est du bon côté.
- Toutes les masses (GND) sont reliées : LED, buzzer, servo, bouton, LCD.
- Le 5V arrive bien au servomoteur et au LCD.
- Les numéros de broches du circuit correspondent à ceux du code (2, 3, 4, 5, 9, 7, 20, 21).

# 5. Le programme

## 5.1 Bibliothèques utilisées

- **Wire.h :** gère la communication I2C avec l'écran LCD.
- **LiquidCrystal_I2C.h :** permet d'écrire sur l'écran LCD. Elle doit être ajoutée dans le gestionnaire de bibliothèques de Wokwi.
- **Servo.h :** permet de commander l'angle du servomoteur (incluse par défaut).

## 5.2 Explication des parties du code

| Partie | Rôle |
|---|---|
| Constantes de broches | Associent un nom lisible (LED_ROUGE, BUZZER…) à chaque numéro de broche. Le code reste facile à modifier. |
| afficher() | Efface le LCD puis écrit deux lignes de texte : une confirmation visible pour l'utilisateur. |
| setLed() | Allume ou éteint une LED, affiche la confirmation sur le LCD et envoie le message dans le moniteur série. |
| setBuzzer() | Active le buzzer avec tone(BUZZER, 1000) (son de 1000 Hz) ou l'arrête avec noTone(). |
| setServo() | Envoie l'angle demandé au servomoteur avec servo.write() et confirme. |
| toutEteindre() | Éteint les trois LED, arrête le buzzer et confirme. Appelée quand on appuie sur le bouton. |
| traiter() | Reçoit le texte, le nettoie (trim) et le met en majuscules (toUpperCase), puis le compare à chaque commande connue. |
| setup() | S'exécute une fois au démarrage : initialise le moniteur série, les broches, le servo (90°) et le LCD. |
| loop() | S'exécute en boucle : lit les commandes reçues et surveille le bouton. |

## 5.3 Points techniques importants

- **INPUT_PULLUP :** la broche du bouton vaut HIGH au repos et LOW lorsqu'on appuie. C'est pourquoi le code teste « == LOW ».
- **readStringUntil('\n') :** lit les caractères reçus jusqu'au retour à la ligne, ce qui correspond à la touche Entrée.
- **delay(300) :** anti-rebond. Il évite qu'un seul appui sur le bouton soit compté plusieurs fois.
- **Adresse du LCD :** 0x27 dans le code. Si l'écran reste vide, essayer 0x3F.

## 5.4 Code complet

Le code ci-dessous est à copier dans le fichier sketch.ino du projet Wokwi.

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <Servo.h>
```

```
const int LED_ROUGE = 2, LED_VERTE = 3, LED_JAUNE = 4;
const int BUZZER = 5, SERVO_PIN = 9, BOUTON = 7;

LiquidCrystal_I2C lcd(0x27, 16, 2);
Servo servo;

void afficher(String l1, String l2 = "") {
  lcd.clear();
  lcd.setCursor(0, 0); lcd.print(l1);
  lcd.setCursor(0, 1); lcd.print(l2);
}

void setLed(int pin, bool etat, String nom) {
  digitalWrite(pin, etat ? HIGH : LOW);
  afficher(nom, etat ? "ALLUMEE" : "ETEINTE");
  Serial.println(nom + (etat ? " allumee" : " eteinte"));
}

void setBuzzer(bool on) {
  if (on) tone(BUZZER, 1000); else noTone(BUZZER);
  afficher("Buzzer", on ? "SONNE" : "ARRETE");
  Serial.println(on ? "Buzzer actif" : "Buzzer arrete");
}

void setServo(int angle) {
  servo.write(angle);
  afficher("Servo", "Angle: " + String(angle));
  Serial.println("Servo a " + String(angle) + " degres");
}

void toutEteindre() {
  digitalWrite(LED_ROUGE, LOW);
  digitalWrite(LED_VERTE, LOW);
  digitalWrite(LED_JAUNE, LOW);
  noTone(BUZZER);
  afficher("Bouton appuye", "Tout eteint");
  Serial.println("Bouton : LED eteintes et buzzer arrete");
}

void traiter(String cmd) {
  cmd.trim();
  cmd.toUpperCase();

  if      (cmd == "ROUGE ON")  setLed(LED_ROUGE, true,  "LED rouge");
  else if (cmd == "ROUGE OFF") setLed(LED_ROUGE, false, "LED rouge");
  else if (cmd == "VERTE ON")  setLed(LED_VERTE, true,  "LED verte");
  else if (cmd == "VERTE OFF") setLed(LED_VERTE, false, "LED verte");
  else if (cmd == "JAUNE ON")  setLed(LED_JAUNE, true,  "LED jaune");
  else if (cmd == "JAUNE OFF") setLed(LED_JAUNE, false, "LED jaune");
  else if (cmd == "SERVO 0")   setServo(0);
  else if (cmd == "SERVO 90")  setServo(90);
  else if (cmd == "SERVO 180") setServo(180);
  else if (cmd == "BUZZER ON") setBuzzer(true);
  else if (cmd == "BUZZER OFF") setBuzzer(false);
  else if (cmd.length() > 0) {
    afficher("Commande", "inconnue");
    Serial.println("Commande inconnue : " + cmd);
  }
}

void setup() {
  Serial.begin(9600);
```

```
  pinMode(LED_ROUGE, OUTPUT);
  pinMode(LED_VERTE, OUTPUT);
  pinMode(LED_JAUNE, OUTPUT);
  pinMode(BUZZER, OUTPUT);
  pinMode(BOUTON, INPUT_PULLUP);
  servo.attach(SERVO_PIN);
  servo.write(90);
  lcd.init();
  lcd.backlight();
  afficher("Assistant pret", "Tapez une commande");
  Serial.println("Commandes : ROUGE/VERTE/JAUNE ON|OFF, SERVO 0|90|180, BUZZER
ON|OFF");
}

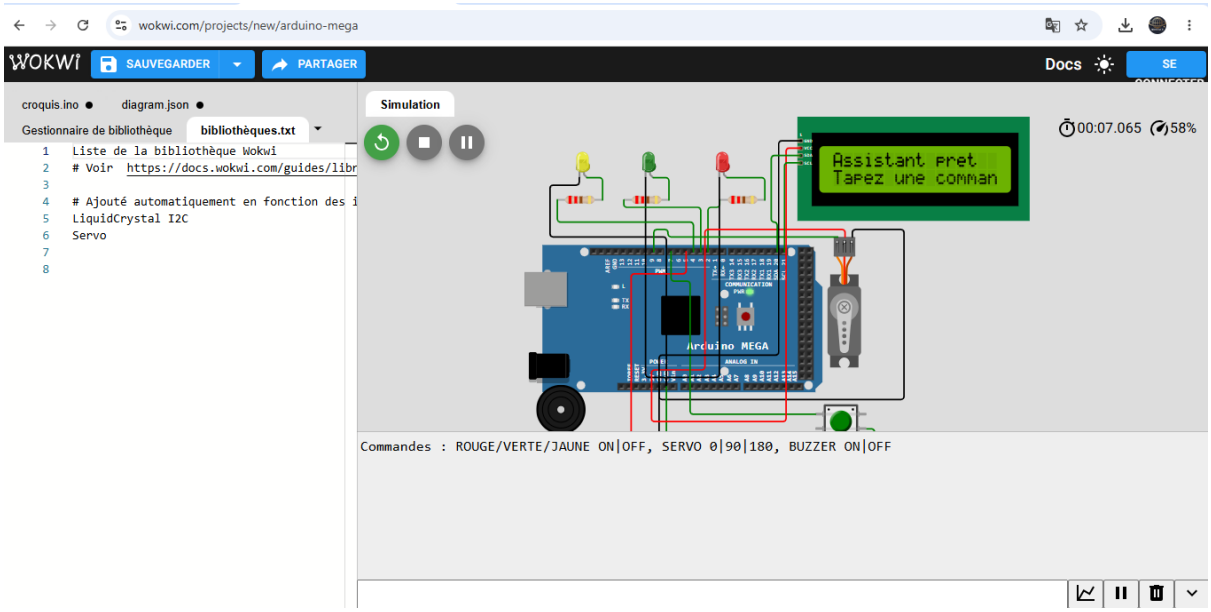
void loop() {
  if (Serial.available()) {
    String cmd = Serial.readStringUntil('\n');
    traiter(cmd);
  }
  if (digitalRead(BOUTON) == LOW) {
    toutEteindre();
    delay(300);  // anti-rebond
  }
}
```

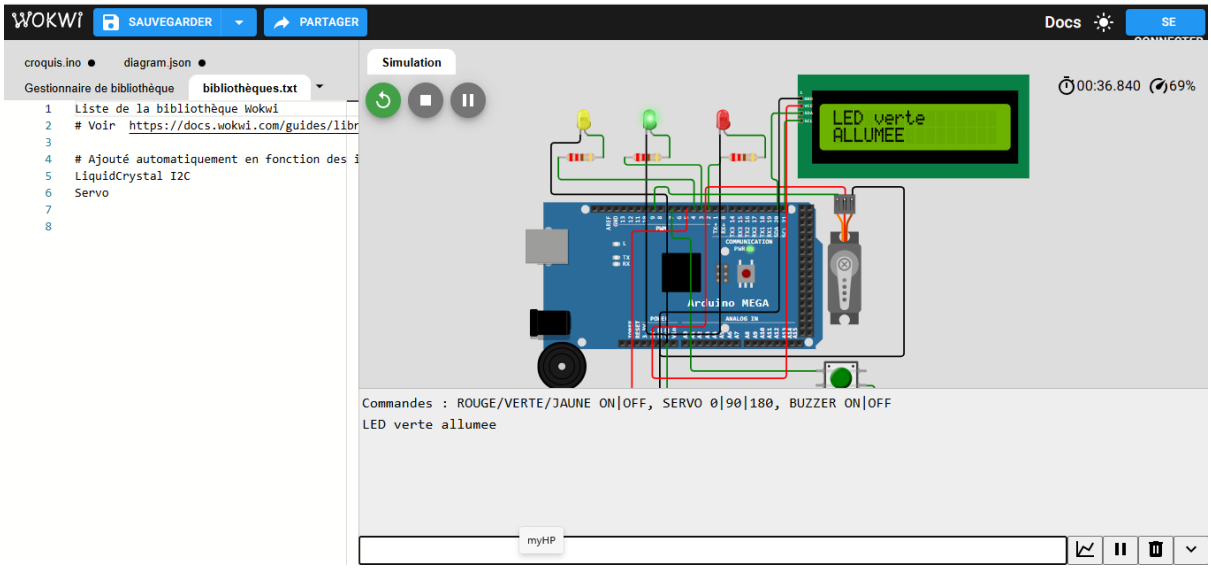
## 6. Plan de tests

Pour chaque test, une capture d'écran doit montrer simultanément le circuit, l'écran LCD et le moniteur série.

| N° | Commande / action | Résultat attendu |
|---|---|---|
| 1 | ROUGE ON | LED rouge allumée ; LCD : « LED rouge / ALLUMEE ». |
| 2 | VERTE ON | LED verte allumée ; LCD : « LED verte / ALLUMEE ». |
| 3 | JAUNE ON | LED jaune allumée ; LCD : « LED jaune / ALLUMEE ». |
| 4 | ROUGE OFF, VERTE OFF, JAUNE OFF | Chaque LED s'éteint et le LCD confirme. |
| 5 | SERVO 0 | Le servo se place à 0° ; LCD : « Servo / Angle: 0 ». |
| 6 | SERVO 90 | Le servo se place à 90°. |
| 7 | SERVO 180 | Le servo se place à 180°. |
| 8 | BUZZER ON | Le buzzer sonne ; LCD : « Buzzer / SONNE ». |
| 9 | BUZZER OFF | Le buzzer s'arrête ; LCD : « Buzzer / ARRETE ». |
| 10 | Allumer une LED et le buzzer, puis appuyer sur le bouton | Tout s'éteint ; LCD : « Bouton appuye / Tout eteint ». |
| 11 | BLEU ON (commande invalide) | LCD : « Commande / inconnue » ; message dans le moniteur série. |

## 7. Simulation en image




Lien du travail : https://wokwi.com/projects/477447164871324673